

Rule 0.0.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust compiler checks for unreachable code, warnings will be generated. Unused `Result` types (Rust's error type) will also generate a warning, as do unused variables in general. Unused variable warnings can be promoted to compiler errors. (<https://doc.rust-lang.org/rustc/lints/listing/warn-by-default.html#unreachable-code>)

```
1  #[deny(unreachable_code, unused_variables)]
2
3  fn return_res(change_me: &mut u64) -> Result<(), ()> {
4      *change_me = 5;
5      Err(())
6  }
7  fn main() {
8      let mut change_me = 0;
9      let unused_result = return_res(&mut change_me);
10     print!("change_me: {}", change_me);
11     return;
12     print!("This generates an unreachable code warning");
13 }
```

Listing 1: Rust Code Example for guideline 0.0.1

Rule 0.0.2

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Rust does not prevent invariant controlling expressions (see example `if`, `while`), and does generally not give a warning. Rust can emit warnings if the comparison is useless due to type limits (<https://play.rust-lang.org/?version=stable&mode=debug&edition=2024&gist=57651b13a36c6e24c90ee9051277194f>). Regarding `for`-loops: Rust does act on ranges, so there is no controlling expression.

```
1  fn main() {
2      let a = 1;
3      if a == 2 {
4          print!("Hello, this should never reach");
5      }
6
7      while a == 1 {
8          print!("Hello infinite loop");
9      }
```

10 }

Listing 2: Rust Code Example for guideline 0.0.2

Rule 0.1.1

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Rust compiler gives warning for values that are assigned, but not read. Unused variables (which are destroyed before being observed) also throw a warning. Writing to values in a loop, before reading or observing them, does not trigger a warning. Therefore the rule is not exhaustively handled. Both warnings can be promoted to compilation errors.

```
1  #![deny(unused_assignments, unused_variables)]
2
3  fn get_num() -> i64 {
4      return 5;
5  }
6
7  fn main() {
8      let mut double_write_before_read = get_num();
9      // write again before reading, throws warning during
      // compilation!
10     double_write_before_read = 10;
11     println!(
12         "I was unnecessarily written to: {}",
13         double_write_before_read
14     );
15
16     // No warning, although an overwrite occurs before reading/
      // observing the incremented value in line 18
17     let mut increment_without_effect = 0;
18     for _ in 0..10 {
19         increment_without_effect = 10;
20         increment_without_effect += 1;
21     }
22     println!(
23         "I was incremented without effect: {}",
24         increment_without_effect,
25     );
26
27     {
28         // causes unused variable compilation warning
29         let i_just_get_destroyed = get_num();
30         let me_too: [u8; 10] = [0; 10];
31     }
32
33     // No warning
34     let i_get_observed_and_read = get_num();
35     let observer = i_get_observed_and_read;
```

```

36     println!("I was read and observed: {}", observer);
37 }

```

Listing 3: Rust Code Example for guideline 0.1.1

Rule 0.1.2

Analysis Result

(C6) Rule still required (safe Rust)

Comment: For a general unused return value, there is no warning given. Rust's error type `Result` produces an explicit compiler warning if its returned values are unused. Rust provides a language feature for annotating custom functions as `#[must_use]`, requiring callers to use the return value. If not, a compiler warning is emitted. Additionally, the standard library provides `#[must_use]` annotations for critical functions: <https://std-dev-guide.rust-lang.org/policy/must-use.html>. The warnings can be promoted to a compilation error.

```

1  #![deny(unused)]
2
3  fn return_res(change_me: &mut u64) -> Result<i32, ()> {
4      *change_me = 5;
5      Err(())
6  }
7
8  fn return_something(change_me: &mut u64) -> u64 {
9      *change_me = 5;
10     7
11 }
12
13 fn main() {
14     let mut change_me = 0;
15     // Throws warning for unused Result
16     return_res(&mut change_me);
17     // No warning for unused u64
18     return_something(&mut change_me);
19 }

```

Listing 4: Rust Code Example for guideline 0.1.2

Rule 0.2.1

Analysis Result

(C3) Rule can be dropped completely

Comment: In Rust, declared variables have to be initialized before use. Unused variables throw a compilation warning, which is promotable to a compilation error (See example for rule 0.0.1, 'Rust Reference: Variables p. 375/432').

Rule 0.2.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Per the rationale, this rule is specifically relevant when providing implementations for virtual functions in C++. As Rust does not have a directly comparable concept, the same reasoning as for rule 0.2.1 applies. Unused variables will give a compiler warning in any case. As a remark, the compiler directive `#[allow(unused)]` exists in Rust, which is comparable to `[[maybe_unused]]` in C++. Usage of this directive should be addressed in future Rust guidelines. The warning is promotable to a compilation error.

Rule 0.2.3

Analysis Result

(C3) Rule can be dropped completely

Comment: Unused type alias produces compilation warning. Rust does not have an equivalent for an unnamed namespace, the equivalent for a namespace is a module. Unused type aliases in a named module also throw a compilation warning. The warnings are promotable to compilation errors.

```
1  #![deny(unused)]
2
3  mod name_space {
4      // (named) module (equivalent to namespace in C++)
5      // unused type alias warning
6      pub type DeclaredWithPublicVisibility = u64;
7  }
8
9  fn main() {
10     {
11         // Block scope
12         // unused type alias warning
13         type DeclaredWithLimitedVisibility = u64;
14     }
15     println!("Type was not used");
16 }
```

Listing 5: Rust Code Example for guideline 0.2.3

Rule 0.2.4

Analysis Result

(C3) Rule can be dropped completely

Comment: Functions with non-public visibility in structs and non-public modules with functions of any visibility produce warnings when unused. The warnings are promotable to compilation errors.

```

1 mod name_space {
2     // (named) module (equivalent to namespace in C++)
3     // unused function produces compiler warning
4     pub fn unused_fn() {}
5 }
6
7 pub struct MyStruct {
8     // public struct
9     pub field: i32,
10 }
11
12 impl MyStruct {
13     // does produce an unused warning
14     fn unused_fn() {}
15
16     // does not produce an unused warning, because the visibility
17     // is public.
18     pub fn unused_member_fn(self) {}
19 }
20
21 fn main() {
22     let _ = MyStruct { field: 0 };
23 }

```

Listing 6: Rust Code Example for guideline 0.2.4

Dir 0.3.1

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Rust does not provide suspicious (imprecise) use warnings. Signalling for infinities exist, but the rationale behind the directive is not satisfied. Rust supports floating-point arithmetic, but appropriate usage depends on context and application-level constraints. By default Rust uses 64-bit floating-point numerical types, which lessens the accumulation of precision errors in common use cases.

```
1 fn main() {
2     let f1: f32 = 1e38_f32;
3     let f2: f32 = 5.2343249027384902374902374902378492034890;
4     let f3: f32 = 0.00001234199899999999;
5 }
```

[illegible]

Listing 7: Rust Code Example for guideline 0.3.1

Dir 0.3.2

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Naturally applies, as there can be arbitrary preconditions for any function defined by a programmer. Rust does not enforce preconditions automatically, but the type system and traits support this via `Options`, `Results` and the newtype pattern: <https://play.rust-lang.org/?version=stable&mode=debug&edition=2024&gist=7a2572eed534710a62a2572d980e728a>.

Rule 4.1.1

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: No comparable standard exists for the Rust programming language yet, but should it exist, this rule would be required. Active work is being done: <https://github.com/rust-lang/rust/issues/113527>, <https://github.com/rust-lang/reference>.

Rule 4.1.2

Analysis Result

(C6) Rule still required (safe Rust)

Comment: This rule naturally applies to the Rust programming language. Details in the rationale refer to specific C++ standards to identify deprecated

features. Deprecated is already defined in rust: <https://rust-lang.github.io/rfcs/1270-deprecation.html>. The attribute emits a compilation warning by default, which can be promoted to a compilation error. See example.

```
1  #![deny(deprecated)]
2
3  #[deprecated]
4  fn a() {
5
6  }
7
8  fn main() {
9      a();
10 }
```

Listing 8: Rust Code Example for guideline 4.1.2

Rule 4.1.3

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: This rule naturally applies to code in safe Rust. Such code either works or panics. A non exhaustive list of behaviour considered undefined in Rust can be found in ‘Rust Reference, page 404/432’.

Rule 4.6.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: The Rust borrow checker guarantees that only one mutable reference to any memory location exists at one point in time. This naturally enforces a sequencing of accesses. However, unsafe Rust still requires manual sequence awareness by developers. See ‘Rust Programming Language: Mutable References, Page 92/626’.

Rule 5.0.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not have a trigraph equivalent.

Rule 5.7.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust expects a closing marker for every opened comment block with `/`. In case there is not a matching number of closing markers, the Rust compiler throws an error.

```
1  /* comment ommitting comment end marker, critical_fn gets included
   in comment by mistake
2  pub fn critical_fn() {}
3
4  /* another comment, unrelated to the first one */
5
6  fn main() {
7      print!("Hello World");
8  }
9
10 // Throws unterminated block comment error error[E0758], as every
    opened comment block is expected to have a closing marker
```

Listing 9: Rust Code Example for guideline 5.7.1

Dir 5.7.2

Analysis Result

(C6) Rule still required (safe Rust)

Comment: This directive naturally applies for Rust as well, as there is no restriction on code in comments.

Rule 5.7.3

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust does not have a line-splicing syntax like C++. The line-splicing escape symbol only has effect within strings, comments are unaffected, as demonstrated in the example.

```

1 fn main() {
2     //Line splicing comment, that does not impact the next line: \
3     println!("I am unaffected by the \\ in the line above");
4     println!(
5         "Multiline strings can use the escape character \\ to e.g.
6         \
7         do a linebreak without breaking the string"
8     );
9 }

```

Listing 10: Rust Code Example for guideline 5.7.3

Rule 5.10.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust, like C++, conforms to Unicode definitions to allow identifiers as described in ‘Rust Reference: Identifiers p. 12/432’. Identifiers are not allowed to have a reserved name, except when marked with the `r#` prefix, and the usable character set is restricted in certain syntactical positions. The intended restriction the MISRA rule enforces is already enforced due to the identifier definition and syntax choice of Rust.

Rule 5.13.1

Analysis Result

(C3) Rule can be dropped completely

Comment: The grammar accepted by the lexer for character and string literals is given in ‘Rust Reference p. 22/432’. Any expression not conforming to these rules will/should give a compilation error.

```

1 fn main() {
2     // Compilation error: unknown character escape
3     let _ = "hello \k world";
4 }

```

Listing 11: Rust Code Example for guideline 5.13.1

Rule 5.13.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust does not support octal escape sequences inside of string literals. Rust enforces termination, as can be seen in the example. For hexadecimal escape sequences in Rust, the lexer only accepts the literal in pairs of two. Therefore, the rationale does not apply anymore.

```
1 fn main() {  
2     // Compilation error: unknown character escape  
3     println!("Unicode escape requires braces: \u{001F6}00, clearly  
4         showing the termination of the escape sequence");  
5     // Hexadecimal Escape Sequence does not have a clear  
6     termination -> Non-compliant  
7     println!("\x41BCD");  
8 }
```

Listing 12: Rust Code Example for guideline 5.13.2

Rule 5.13.3

Analysis Result

(C3) Rule can be dropped completely

Comment: The rationale for the rule is to avoid confusion, as in C++ a leading 0 marks an octal constant. In Rust, the leading 0 is ignored. To create an octal value, the 0o prefix is required, leaving no room for confusion.

```
1 fn main() {  
2     let not_an_octal = 0345;  
3     println!("Prints 345: {}", not_an_octal);  
4  
5     let an_octal = 0o345;  
6     println!("\nPrints 229: {}", an_octal);  
7 }
```

Listing 13: Rust Code Example for guideline 5.13.3

Rule 5.13.4

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust supports explicit suffixing of unsigned integer literals to specify their type. Common suffixes include u8, u16, u32, u64, u128, and usize. However, Rust does not have automatic integer promotion and its type system is platform-consistent. The rationale of this rule does not apply anymore.

```
1 fn main() {
2     // Unsigned integer literals with appropriate suffixes
3     let small_number: u8 = 42u8;
4     let medium_number: u32 = 1000u32;
5     let large_number: u64 = 1_000_000u64;
6
7     println!("Small number: {}", small_number);
8     println!("Medium number: {}", medium_number);
9     println!("Large number: {}", large_number);
10
11     // defaults to i32 without type hint in a 64bit environment
12     let without_suffix = 55;
13 }
```

Listing 14: Rust Code Example for guideline 5.13.4

Rule 5.13.5

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust does not have a L suffix to denote long integers. Only explicit suffixes are used (u64, i64).

```
1 fn main() {  
2     // Does not compile, Rust does not have L suffix to denote long  
    integers  
3     let some_long = 5L;  
4  
5     // Rust uses explicit suffixes to denote integer types  
6     let signed_long = 1000000000000000000000i64;  
7     let unsigned_long = 1000000000000000000000u64;  
8 }
```

Listing 15: Rust Code Example for guideline 5.13.5

Rule 5.13.6

Analysis Result

(C3) Rule can be dropped completely

Comment: See explanation for rule 5.13.5, Rust uses explicit suffixes for long integers.

Rule 5.13.7

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust exposes only one String type (UTF8-encoded) in the standard library ('Rust Programming Language: What is a String? p.181/626'), as a String literal denoted as `".."`. Furthermore Raw string literals, Byte string literals, raw byte string literals and character literals exist ('Rust Programming Language, Table B-2 p. 608/626'), which are explicit in their type and can not be concatenated directly (see example).

```
1 fn main() {
2     // This will not compile because different types of string
3     // literals cannot be concatenated
4     //let concatenated = "Hello, " + r"world!"; // Error:
5     //      mismatched types
6
7     // Correct usage: concatenate same type of string literals
8     let concatenated = "Hello, ".to_string() + "world!";
9     println!("{}", concatenated);
10
11    // Correct usage: concatenate raw string literals
12    let raw_concatenated = r"Hello, ".to_owned() + r"world!";
13    println!("{}", raw_concatenated);
14
15    // Correct usage: concatenate byte string literals
16    let byte_concatenated = b"Hello, ".to_vec().extend_from_slice(b
17        "world!");
18    println!("{}", byte_concatenated);
19 }
```

Listing 16: Rust Code Example for guideline 5.13.7

Rule 6.0.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Function definitions in block scope are possible, but free functions always require a body in Rust, not just a declaration. Furthermore due to the syntax definition in Rust, no ambiguity as addressed by the rule can be forced in Rust (see example).

```
1 type A = i32;
2
3 fn main() {
4     {
```

```

5      fn test() -> A {
6          42
7      }
8      let _some_var: A = 42;
9      test();
10 }
11 }

```

Listing 17: Rust Code Example for guideline 6.0.1

Rule 6.0.2

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Rust always requires explicit size for an array, external linkage included (see example).

```

1  // not possible to have an external linkage of an array without
   // explicit size
2  // unsafe extern "C" {
3  //     // compiler error: missing type for static
4  //     static EXTERNAL_ARRAY;
5  // }
6
7  unsafe extern "C" {
8      // no compiler error with explicit size
9      static _EXTERNAL_ARRAY: [u8; 10];
10 }
11
12 fn main() {}

```

Listing 18: Rust Code Example for guideline 6.0.2

Rule 6.0.3

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: Part of the rationale for this rule is the influence of include ordering on program behaviour. Due to strict rules of unambiguous names when using crates in Rust, this rule is partly satisfied. Prevention of cluttering during lookup still requires following this rule in Rust code (see example). A potential adaptation could involve the usage of ‘prelude’ modules that are often used within the Rust ecosystem.

```

1 // function without namespace
2 pub fn non_compliant_function() {
3     println!("Hello from non_compliant_function");
4 }
5
6 mod module_a {
7     pub fn function() {}
8 }
9
10 mod module_b {
11     pub fn function() {}
12 }
13
14 fn main() {
15     use module_a::*;
16     use module_b::*;
17     // compilation error: 'function' is ambiguous
18     function();
19 }

```

Listing 19: Rust Code Example for guideline 6.0.3

Rule 6.0.4

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Use of name `main` is possible in modules, rule also applies for Rust (see example).

```

1 mod non_entrypoint_main {
2     // non-compliant use of name main
3     pub fn main() {}
4 }
5 fn main() {
6     use non_entrypoint_main;
7     non_entrypoint_main::main();
8 }

```

Listing 20: Rust Code Example for guideline 6.0.4

Rule 6.2.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: The explicit typing in Rust prevents the definition of two identical types within the same translation unit (compilation unit a.k.a. crate).

However, some loop holes exist. The usage of the `no_mangle` attribute allows one to define and link against two function versions of the same name. Ultimately, the resulting run-time behavior, as in the case of C/C++, depends on the linker used for the final binary. The attribute is usable within safe Rust until edition 2024. From edition 2024, the attribute is marked `unsafe`. See example 6.2.1.

Rule 6.2.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust does not allow overloading like C++, duplicate names (with or without different arguments) will always result in a compilation error (see example, also example for rule 6.0.4, 6.0.3).

```
1 mod fun_with_int_argument {
2     // non-compliant use of name main
3     pub fn function(a: i32) -> () {}
4 }
5
6 mod fun_with_char_argument {
7     pub fn function(a: char) -> () {}
8 }
9 fn main() {
10     use fun_with_char_argument::*;
11     use fun_with_int_argument::*;
12     // Compilation error: 'function' is ambiguous
13     function(3);
14 }
```

Listing 21: Rust Code Example for guideline 6.2.2

Rule 6.2.3

Analysis Result

(C3) Rule can be dropped completely

Comment: Multiple definitions of the same entity always result in a compilation error. The rule allows identical definitions in separate translation units. In Rust even this will cause a compilation error, when ambiguous definitions are imported/used.

```

1 type A = i32;
2 // Compilation error: defined multiple times
3 type A = i32;
4
5 static global_i: i32 = 4;
6
7 // Compilation error: defined multiple times
8 static global_i: i32 = 4;
9
10 const GLOBAL_K: i32 = 5;
11 // Compilation error: defined multiple times
12 const GLOBAL_K: i32 = 5;
13
14 pub trait MyTrait {}
15 // Compilation error: defined multiple times
16 pub trait MyTrait {}
17 fn main() {}

```

Listing 22: Rust Code Example for guideline 6.2.3

Rule 6.2.4

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust has no notion of header files. This rule’s goal is to prevent violations of the one-definition rule. This is possible in unsafe Rust (see 6.2.1).

Rule 6.4.1

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Rust allows shadowing in inner scopes (see example). However, the original rationale mentions developer confusion as the reason for the rule. Rust style guidelines advocate for variable rebinding as a form of clean code style. Additionally, global variable names are uppercase snakecase names and checked using lints.

```

1 fn main() {
2     // gives unused variable warning
3     let shadowed_j = 1;
4     {
5         let shadowed_j = 2;
6         //prints 2
7         println!("inner_j: {}", shadowed_j);
8     }

```

```
9 }
```

Listing 23: Rust Code Example for guideline 6.4.1

Rule 6.4.2

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: While Rust supports some form of inheritance using traits, Rust does not allow arbitrary method concealing. Traits fundamentally work by exposing all methods that they define on their implementation type. Structs can only shadow trait methods, but can never fully conceal them to consumers (see second example). Rust does not provide the possibility to have two functions with the same name and different parameter types. However, in a strict sense, the same behavior is present in Rust as it is in C++, due to the way traits work in general.

```
1 trait Base {
2     fn base_fn(&self) -> () {
3         println!("Base function");
4     }
5 }
6 struct DerivedBase;
7 impl Base for DerivedBase {}
8
9 struct DerivedOverwrite;
10 impl Base for DerivedOverwrite {
11     fn base_fn(&self) -> () {
12         println!("Derived function");
13     }
14 }
15
16 fn main() {
17     let derived_base = DerivedBase;
18     let derived_overwrite = DerivedOverwrite;
19     // Prints: Base function
20     derived_base.base_fn();
21     // Prints: Derived function
22     derived_overwrite.base_fn();
23 }
```

Listing 24: Rust Code Example for guideline 6.4.2

Rule 6.4.3

Analysis Result

(C3) Rule can be dropped completely

Comment: As Rust handles resolution of names in an explicit way with a hierarchical order, any kind of unqualified lookup is not possible Rust Reference: Namespaces p. 350/432’ (also see reasoning for rules 6.0.2, 6.0.3, 6.0.4, 6.2.1).

Rule 6.5.1

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: Rust has no notion of header files. External linkage has to be explicitly typed, see ‘Rust Reference: Functions p. 118/432’. Accessing the functions and objects defined with external linkage (such as `extern "C"`) is only possible within an `unsafe` context since edition 2024. However, similar behavior to headers can be achieved with Rust modules and using public visibility modifiers.

```
1 unsafe extern "C" {
2     fn external_function(x: i32) -> i32;
3 }
4 unsafe extern "C" {
5     static EXTERNAL_VARIABLE: i32;
6 }
7 fn main() {
8     // Compilation error: call to unsafe function must be inside
9     // unsafe block
10    //let result = external_function(42);
11
12    // Would compile, if external linkage existed
13    unsafe {
14        let result = external_function(42);
15        println!("Result: {}", result);
16    }
```

Listing 25: Rust Code Example for guideline 6.5.1

Rule 6.5.2

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: It is possible to specify linkage in a non-compliant way (see example, unsafe since 2024).

```

1 mod some_namespace {
2     // non-compliant: external linkage within a namespace
3     unsafe extern "C" {
4         pub fn external_function(x: i32) -> i32;
5     }
6 }
7
8 fn main() {
9     // Would compile, if external linkage existed
10    unsafe {
11        let result = some_namespace::external_function(42);
12        println!("Result: {}", result);
13    }
14 }

```

Listing 26: Rust Code Example for guideline 6.5.2

Rule 6.7.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: The rationale is possible race conditions. `mut static` variables in Rust can not be changed without an `unsafe` context. So possible Race conditions in a safe Rust context are guarded against, and the restriction on placement of static variables are not necessary (see example).

```

1 fn main() {
2     static mut test_var: i32 = 0;
3     // Compilation error: use of mutable static requires unsafe
4     // function or block
5     test_var += 1;
6 }

```

Listing 27: Rust Code Example for guideline 6.7.1

Rule 6.7.2

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rationale is uncontrolled mutability of global variables from arbitrary parts of the program, as well as unspecified initialization. See reasoning for rule 6.7.1, in Rust mutable global variables cannot be accessed without an `unsafe` block. For safe access, mutual exclusion mechanisms are enforced via the `Send` trait. However, the implementation of the `Send` trait can be done for arbitrary, custom data types within an `unsafe` context. Still, global variables

can increase program complexity/unpredictability as described in the rationale.

Rule 6.8.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust's strong lifetime checks prevent an access after end of life. Any references require lifetime annotations, unless lifetime elision is possible. In that way the compiler can verify that objects are not accessed after their lifetime ends (see 'Rust Reference: Lifetime bounds p. 323/432', 'Lifetime elision p. 339/432', 'Rust Programming Language: Validating References with Lifetimes p. 243/626ff'). `unsafe` blocks do not bypass lifetime checks, but they do allow for raw pointer operations, resulting in a necessary application of this rule in unsafe Rust. See example.

```
1 unsafe fn no_lifetime_extension_within_unsafe_possible(x: &i32) ->
2     &'static i32 {
3     // Fails to compile.
4     // unsafe {
5     //     x
6     // }
7
8     &5 // comment out to check.
9 }
10
11 unsafe fn bypass_borrowing_rules(x: &i32) -> *const i32 {
12     x as *const i32
13 }
14
15 fn main() {
16     unsafe {
17         let _ = no_lifetime_extension_within_unsafe_possible(&1);
18     }
19
20     let ptr: *const i32;
21     {
22         let value = 5;
23         unsafe {
24             ptr = bypass_borrowing_rules(&value);
25         }
26     }
27     unsafe {
28         // miri returns UB due to dangling pointer dereference.
29         println!("{}", *ptr);
30     }
31 }
```

Listing 28: Rust Code Example for guideline 6.8.1

Rule 6.8.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust requires a lifetime annotation for returned references (which can never be valid for local variables), and additionally prohibits return of references to local variables. Returning a dangling pointer is safe to do in Rust, but results in a warning. Dereferencing the pointer is unsafe and will cause UB (see example). The warning can be promoted to a compilation error.

```
1 // compilation error 1: Expected lifetime for return parameter, e.g
  . &'a u64
2 // fn invalid_return() -> &u64 {
3 //     let x = 42;
4 //     // compilation error 2: Cannot return reference to local
      variable
5 //     &x
6 // }
7
8 fn invalid_return_ptr() -> *const u64 {
9     let x = 42u64;
10    // warning: a dangling pointer will be produced because the
      local variable 'x' will be dropped
11    &x as *const u64
12 }
13
14 fn main() {}
```

Listing 29: Rust Code Example for guideline 6.8.2

Rule 6.8.3

Analysis Result

(C3) Rule can be dropped completely

Comment: Due to compiler checks of lifetime, assigning references without compatible lifetimes is not allowed (see example, also example and reasoning for rule 6.8.2, 6.8.1).

```
1 fn main() {
2     let mut x: &i32 = &42;
3     {
4         let local = 42;
5         // Compilation error: 'local' does not live long enough
6         x = &local;
7     }
8     print!("{}", x);
9 }
```

Rule 6.8.4

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: C++ distinction between methods callable on *lvalue* and *rvalue* objects does not directly correspond to a mechanism in Rust. The rationale of preventing dangling references is covered by lifetime checks, for member and non-member functions alike. (see rule 6.8.1). Some checks are in place for unsafe Rust: <https://doc.rust-lang.org/rustc/lints/listing/warn-by-default.html#dangling-pointers-from-locals>.

Rule 6.9.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Multiple definitions with the same name are not allowed and overloading does not exist (see example).

```
1 type A = i32;
2 static some_var: i32 = 42;
3 unsafe extern "C" {
4     // Compilation error: 'some_var' is defined multiple times
5     static some_var: A;
6 }
7
8 fn main() {}
```

Listing 31: Rust Code Example for guideline 6.9.1

Rule 6.9.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust integer types for the most part include their size in the type name. The only exception is the type `usize` and `isize` ('Rust Programming Language: Integer Types p. 48/626').

Rule 7.0.1

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Rust has explicit conversion from type `bool` (see ‘Rust Reference: Type cast expressions p. 205/432’).

```
1 fn main() {  
2     let b: bool = false;  
3     let i: i32 = b as i32;  
4     println!("i: {}", i);  
5 }
```

Listing 32: Rust Code Example for guideline 7.0.1

Rule 7.0.2

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust has no implicit or explicit conversion to type `bool` (see ‘Rust Reference: Type cast expressions p. 205/432’).

Rule 7.0.3

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Explicit cast from `char` to an integer type and the reverse is possible (‘Rust Reference: Type cast expressions p. 205/432’). However, the value of a `char` is well-defined and unicode-safe. The check can be circumvented in unsafe Rust.

Rule 7.0.4

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust enforces matching left- and right-hand types during compilation. It is not possible to shift with values greater than the *lvalue* type.

```

1 fn main() {
2     let b: i8 = 1;
3     let c = b << 256;
4     // does not compile
5     println!("c = {}", c);
6 }

```

Listing 33: Rust Code Example for guideline 7.0.4

Rule 7.0.5

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust does not have a comparable integral promotion like C++ from smaller to larger integer types. Promotion has to be explicit (see example).

```

1 fn main() {
2     let b: u8 = 1;
3     let c: i8 = -2;
4     // error: cannot add 'i8' to 'u8'
5     let d = b + c;
6     // error: mismatched types, expected 'i8' for variable b, but
7     // found 'u8'
8     let e = c > b;
9
10    // Explicit cast allowed
11    let f = b + c as u8;
12    // Prints: f = 255
13    print!("f = {}", f);
14 }

```

Listing 34: Rust Code Example for guideline 7.0.5

Rule 7.0.6

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Compile time assignments are checked for appropriateness, during runtime assignments with e.g. overflow will result in a panic if running in debug mode (see example).

```

1 fn main() {
2     // compile time initialization is checked:
3     // Compilation error: literal out of range for 'u8'
4     //let not_possible: u8 = 300;
5 }

```

```

5
6   let mut counter: u8 = 0;
7   for i in 0..256 {
8       // Will panic during runtime in debug mode only:
9       // attempt to add with overflow
10      counter += 1;
11  }
12  println!("Counter: {}", counter);
13 }

```

Listing 35: Rust Code Example for guideline 7.0.6

Rule 7.11.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust does not have the notion of raw pointers in a non-**unsafe** context, including the `nullptr` ('Rust Reference: Raw Pointers p. 289/432'). Its equivalent, `std::ptr::null`, is not of type integer, but uses generics to match its return value.

Rule 7.11.2

Analysis Result

(C3) Rule can be dropped completely

Comment: An array is a fixed-size sequence in Rust, passing as a reference is possible. In contrast to C++, the reference is a *slice* type, not a raw pointer, and bounds are still checked during compilation and runtime 'Rust Reference: Array types, Slice types p. 279/432ff' (see example). Arrays passed over FFI boundaries are out of scope.

```

1 fn takes_array(array: [i32; 3]) {
2     println!("Array: {:?}", array);
3 }
4 fn takes_slice(slice: &[i32]) {
5     println!("Slice: {:?}", slice);
6 }
7
8 fn main() {
9     let array: [i32; 3] = [1, 2, 3];
10    // pass as reference
11    takes_slice(&array);
12    // pass as value
13    takes_array(array);
14 }

```

Rule 7.11.3

Analysis Result

(C3) Rule can be dropped completely

Comment: Function item and function pointer types are explicit in Rust, no ambiguity is possible; ‘Rust Reference: Function item types p. 284/432’. The rationale of the rule does not apply to Rust due to its stricter type system.

Rule 8.0.1

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Rust has less operators and expressions (‘Rust Reference: Expression precedence p. 180/432’), but the rule naturally applies.

Rule 8.1.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rationale is to prevent undefined behaviour, if the lambda is called after the object’s lifetime ends. In Rust, lifetime checks prevent this (see example).

```
1 struct MyStruct {  
2     value: i32,  
3 }  
4  
5 impl MyStruct {  
6     fn create_closure<'a>(&'a self) -> impl Fn() + use<'a> {  
7         // Closure capturing '&self'  
8         move || {  
9             println!("Value: {}", self.value);  
10        }  
11    }  
12  
13    fn change_value(&mut self, value: i32) {  
14        self.value = value;  
15    }  
16 }
```

```

15     }
16 }
17
18 fn main() {
19     let my_struct = MyStruct { value: 42 };
20     let closure = my_struct.create_closure();
21     // Compilation Error: can not drop, because my_struct is still
22     // borrowed (by closure because of moved self)
23     drop(my_struct);
24     closure();
25 }

```

Listing 37: Rust Code Example for guideline 8.1.1

Rule 8.1.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Rationale is to prevent dangling pointers in the lambda (in a Rust closure). Due to lifetime checks, this is not possible in Rust (see rule 8.1.1). The misleading or confusing interpretation problem is solved by Rust’s choice of syntax: the `move` keyword.

Rule 8.2.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: In Rust, the `dyn` keyword can be used to describe trait behaviour without specifying an explicit type at compile time. Cast to a specific type (corresponding to a derived class in C++) can be done with the call `downcast_ref::<DerivedType>`, which corresponds to this rule’s requirements; ‘Rust Programming Language: Defining a Trait for Common Behavior p. 462/626’.

Rule 8.2.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Type casts in Rust are explicit. The rationale for this rule is prevention of implicit or hard to identify type casts should be satisfied by this (see ‘Rust Reference: Type cast expressions p. 205/432’).

Rule 8.2.3

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust enforces strict ownership and borrowing rules that prevent casting from removing immutability or mutability qualifiers ('Rust Reference: Type cast expressions p. 205/432').

Rule 8.2.4

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust enforces strict type safety and does not allow arbitrary casts between function pointers and other types. Only safe and explicit casts are permitted, ensuring that undefined behavior is avoided ('Rust Reference: Type cast expressions p. 205/432').

Rule 8.2.5

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust equivalent would be function `std::mem::transmute`, which can only be used in an `unsafe` context.

Rule 8.2.6

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Cast from pointer to pointer, and pointer to address and reverse are allowed. Furthermore is this section of the reference not fully documented. This rule is still safely handled, as all pointer dereferences have to be in an `unsafe` block ('Rust Reference: Type cast expressions p. 205/432').

Rule 8.2.7

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Interpreting a pointer type as integer will yield that pointer's address in Rust. Accessing a pointer has to be handled in an **unsafe** block ('Rust Reference: Type cast expressions p. 205/432'). The rationale of the rule still holds in **unsafe** context. However, such casts have to reside within **unsafe** blocks, providing developers a strong indication that they have to pay extra attention for the contained code.

Rule 8.2.8

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust has no direct correspondence to specific pointer types, but pointer access is handled in an **unsafe** block ('Rust Reference: Type cast expressions p. 205/432').

Rule 8.2.9

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust's `typeid` equivalent uses a strongly typed approach, that enforces a runtime check, and therefore there is no ambiguity (see example).

```
1 use std::any::{Any, TypeId};
2
3 fn main() {
4     let x: &dyn Any = &42;
5
6     // Rusts equivalent of typeid always forces runtime evaluation
7     if x.type_id() == TypeId::of::<i32>() {
8         println!("x is an i32");
9     } else {
10        println!("x is not an i32");
11    }
12 }
```

Listing 38: Rust Code Example for guideline 8.2.9

Rule 8.2.10

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Recursion is possible, therefore this rule applies (see example).

```
1 fn factorial(n: u32) -> u32 {
2     if n == 0 {
3         1
4     } else {
5         n * factorial(n - 1)
6     }
7 }
8
9 fn main() {
10     let result = factorial(5);
11     println!("Factorial of 5 is: {}", result);
12 }
```

Listing 39: Rust Code Example for guideline 8.2.10

Rule 8.2.11

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust does not have a direct equivalent to ellipsis (variadic functions), as variadic functions are only allowed in external (C-)function blocks ('Rust Reference: Function parameters p.78/432', <https://doc.rust-lang.org/reference/items/external-blocks.html#r-items.extern.v>).

Rule 8.3.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust utilizes only explicit integer promotions, due to the type system applying the `-` operator to an unsigned integer results in a compilation error (see example).

```
1 fn main() {
2     let x: u32 = 42;
3     // Compilation error: cannot apply unary operator '-' to 'u32'
4     let y = -(x as u32);
}
```

```

5 |
6 |     println!("x: {}, y: {}", x, y);
7 | }

```

Listing 40: Rust Code Example for guideline 8.3.1

Rule 8.3.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Rationale for this rule is the integral promotion triggered by the operator. As Rust only has explicit promotion, this rule does not apply (see example for 8.3.1).

Rule 8.7.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Dereferencing a raw pointer is an unsafe operation in Rust, therefore the rationale behind this rule is satisfied ('Rust Reference: Raw Pointers p. 289/432').

Rule 8.7.2

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Operations on raw pointers and dereferencing the result is an unsafe operation in Rust, therefore the rationale behind this rule is satisfied ('Rust Reference: Raw Pointers p. 289/432').

Rule 8.9.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Comparison between pointers is an unsafe operation in Rust, therefore the rationale behind this rule is satisfied ('Rust Reference: Raw Pointers p. 289/432', see example).

```

1 fn main() {
2     let a = 10;
3     let b = 20;
4
5     let a_ptr: *const i32 = &a;
6     let b_ptr: *const i32 = &b;
7
8     if a_ptr < b_ptr {
9         println!("a_ptr is less than b_ptr");
10    } else {
11        println!("a_ptr is not less than b_ptr");
12    }
13 }

```

Listing 41: Rust Code Example for guideline 8.9.1

Rule 8.14.1

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Persistent side effects are possible (see example).

```

1 fn main() {
2     let mut x = 0;
3
4     if true && { x += 1; x > 0 } {
5         println!("x is now {}", x);
6     }
7 }

```

Listing 42: Rust Code Example for guideline 8.14.1

Rule 8.18.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: The Rust equivalent to `std::memcpy` is `std::ptr::copy`, which is semantically equivalent to `std::memmove` per documentation: <https://doc.rust-lang.org/std/ptr/fn.copy.html>.

```

1 fn main() {
2     let mut array = [1, 2, 3, 4, 5];
3     // Compilation error: cannot borrow 'array' as mutable because
4     // it is also borrowed as immutable
5     let src = &array[0..3];

```

```

5  let dst = &mut array[2..5];
6
7  // std::ptr::copy requires unsafe block
8  unsafe {
9      std::ptr::copy(src.as_ptr(), dst.as_mut_ptr(), src.len());
10 }
11 // with copy_from_slice no overlap possible, due to borrow
   rules
12 dst.copy_from_slice(&src);
13
14 println!("{:?}", array);
15 }

```

Listing 43: Rust Code Example for guideline 8.18.1

Rule 8.18.2

Analysis Result

(C3) Rule can be dropped completely

Comment: In contrast to C++, the return type of an assignment is always (). Due to the strict type system, there is no ambiguity (see example).

```

1 fn main() {
2     let mut x = 5;
3     let y = 10;
4     // z is the result of an assignment
5     let z = (x = y);
6
7     assert_eq!(z, ());
8 }

```

Listing 44: Rust Code Example for guideline 8.18.2

Rule 8.19.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Sequencing of expressions uses blocks {} in Rust, the comma operator does not exist. This can be seen as less ambiguous (see example).

```

1 fn times_two(value : i32) -> i32 {
2     value * 2
3 }
4
5 fn main() {
6     let mut x = 5;

```

```

7 // Sequenced expression is clear and concise
8 let y = times_two({x += 1; x});
9 // C++ Syntax:
10 // y = times_two((x += 1, x));
11 assert_eq!(y, 12);
12 }

```

Listing 45: Rust Code Example for guideline 8.19.1

Rule 8.20.1

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Const values are evaluated at compile time in Rust, overflows won't always cause a compilation error – but a panic at runtime. To force wrapping, explicit functions have to be used (see example).

```

1 fn main() {
2     let max_u8: u8 = 255;
3     let wrapped_result = max_u8.wrapping_add(1);
4     // Output: 0
5     println!("Wrapped result: {}", wrapped_result);
6
7     // The following line will cause a compilation error
8     // Compilation error: this arithmetic operation will overflow
9     // let error_no_wrap: u8 = max_u8 + 1;
10 }

```

Listing 46: Rust Code Example for guideline 8.20.1

Rule 9.2.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust syntax for explicit type conversion is unambiguous. The rationale of accidental release of resources due to unclear syntax is solved by this (see example). Resources in Rust are usually dropped by implementing the `Drop` trait. Once a locked `Mutex` is dropped, its resource is released. However, it is syntactically not possible to construct and release such a mutex while causing ambiguity.

```

1 use std::sync::{Arc, Mutex};
2
3 fn main() {
4     let data = Arc::new(Mutex::new());

```

```

5  {
6      // Lock the mutex until the end of this block
7      data.lock().unwrap();
8      // error-prone C++ conversion syntax:
9      // std::scoped_lock { a_mutex };
10     // less ambiguous Rust conversion syntax:
11     // x as U
12 }
13 println!("Mutex is unlocked now");
14 }

```

Listing 47: Rust Code Example for guideline 9.2.1

Rule 9.3.1

Analysis Result

(C3) Rule can be dropped completely

Comment: In Rust, iteration and selection-statements always require curly braces {}, satisfying the rationale of unambiguous compound-statements (‘Rust Reference: Loops and other breakable expressions p. 230/432’).

Rule 9.4.1

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Termination with an `else` statement is not required in Rust (see example).

```

1  fn main() {
2      let x = 10;
3
4      if x < 5 {
5          println!("x is less than 5");
6      } else if x == 5 {
7          println!("x is equal to 5");
8      } // No else block here
9
10     println!("End of program");
11 }

```

Listing 48: Rust Code Example for guideline 9.4.1

Rule 9.4.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust has no `switch` statement, the closest equivalent could be a `match` expression. The rules ensure the complex structure of a switch statement is “well-formed”. Rust `match` statements are required to be exhaustive and have clear pattern matching syntax using logical boolean operators, removing all of the ambiguity.

```
1 fn main() {  
2     let number = 7;  
3  
4     match number {  
5         1 => println!("One"),  
6         2 | 3 => println!("Two or Three"),  
7         4..=6 => println!("Four to Six"),  
8         n if n % 2 == 0 => println!("Even number"),  
9         n if n % 2 != 0 => println!("Odd number"),  
10        _ => println!("Other"),  
11    }  
12 }
```

Listing 49: Rust Code Example for guideline 9.4.2

Rule 9.5.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rationale for this rule is the prevention of unclear number of iterations in a for loop. Due to strict type checks and immutability of the chosen loop index variable, equivalent dangers like in legacy for statements in C++ are not present (see example).

```
1 fn main() {  
2     for i in 0..10 {  
3         // Compilation error: cannot assign twice to immutable  
4         // variable 'i'  
5         // i =5;  
6     }  
7  
8     // Infinite loop  
9     for i in 0.. {  
10        println!("{}", i);  
11    }
```

12 }

Listing 50: Rust Code Example for guideline 9.5.1

Rule 9.5.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: In Rust, the `for` loop initializer typically uses ranges or iterators, which do not utilize function calls in the same manner as a for-range-initializer ('Rust Reference: Predicate pattern loops p. 231/431'). An adaptation of the rule is not required, as the rationale states that the rule shall prevent UB related to object lifetime violations. Due to Rust's borrow checker, these do not occur.

Rule 9.6.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not have a `goto` statement, which inherently satisfies this rule and prevents the associated risks.

Rule 9.6.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: See explanation rule 9.6.1

Rule 9.6.3

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: See explanation rule 9.6.1

Rule 9.6.4

Analysis Result

(C3) Rule can be dropped completely

Comment: In Rust, functions that do not return are indicated with the `!` type. This ensures that the function does not return to the caller. The compiler errors if a function does return instead (see example and ‘Rust Reference: Never type p. 277/432’).

```
1 fn main() {  
2     // Call the function that never returns  
3     never_returning_function();  
4 }  
5  
6 fn never_returning_function() -> ! {  
7     // This function will never return to the caller  
8     loop {  
9         println!("This function will run forever.");  
10    }  
11 }
```

Listing 51: Rust Code Example for guideline 9.6.4

Rule 9.6.5

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust’s strong type system enforces that functions with a non-void return type must return a value on all paths. The Rust compiler checks for this at compile time, ensuring that all code paths in a function return a value of the specified type, even with abstract return types (see example and ‘Rust Reference: Abstract return types p. 297/432’).

```
1 trait Displayable {  
2     fn display(&self);  
3 }  
4  
5 struct TypeA;  
6  
7 impl Displayable for TypeA {  
8     fn display(&self) {  
9         println!("TypeA");  
10    }  
11 }  
12  
13 // Abstract return type of a Trait with missing return value on one  
    path
```

```

14 fn example_function(condition: bool) -> impl Displayable {
15     if condition {
16         return TypeA;
17     }
18     // Compilation error: mismatched types, expected struct 'TypeA
19     //      ', found ()
20 }
21
22 fn main() {
23     example_function(true).display();
24 }

```

Listing 52: Rust Code Example for guideline 9.6.5

Rule 10.0.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not have declaration without initialization before use ('Rust Reference: Variables p. 375/432').

Rule 10.1.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust's mutability rules enforce that references are either mutable or immutable. Immutable references (&T) cannot be used to modify the referenced value, instead explicit `mut` keyword is required (see example and 'Rust Programming Language: Variables and Mutability p. 43/626').

```

1 fn main() {
2     let x = 10;
3     let mut y = 20;
4
5     // Call the function with both immutable and mutable references
6     process_values(&x, &mut y);
7
8     println!("y after processing: {}", y);
9 }
10
11 // Explicit mut keyword is required for mutable references and
12 //      variables
13 fn process_values(immutable_ref: &i32, mutable_ref: &mut i32) {
14     *mutable_ref += *immutable_ref;
15 }

```

Rule 10.1.2

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust's equivalent is `core::ptr::read_volatile`, behaving like in C (https://doc.rust-lang.org/core/ptr/fn.read_volatile.html).

Rule 10.2.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Compared to C++ enums, Rust enums are more flexible, such that each field can have an arbitrary type. This satisfies the rationale of an obvious enum type for the user ('Rust Reference: Enumerations p. 88/432').

Rule 10.2.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust has explicit types and namespaces for enums ('Rust Reference: Enumerations p.88/432').

Rule 10.2.3

Analysis Result

(C3) Rule can be dropped completely

Comment: See explanation for rule 10.2.2

Rule 10.3.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust modules (namespace equivalent) are always named ('Rust Reference: Modules p. 68/432').

Rule 10.4.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust `asm!` macro requires an `unsafe` block to use (see example).

```
1 use core::arch::asm;
2 fn main() {
3     unsafe {
4         asm!("nop");
5     }
6 }
```

Listing 54: Rust Code Example for guideline 10.4.1

Rule 11.3.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust arrays are fixed-size and bound check during compilation. Rust's general move semantics apply, as the `Array` type does not implement the `Copy` trait, thus the rationale of this rule is satisfied ('Rust Reference: Array types p. 278/432', 'Copy Trait: p. 345/432').

Rule 11.3.2

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust allows multiple levels of pointer indirection (see example). While the example code is safe Rust, the usage of pointers via dereferencing

them is an unsafe operation. The multi-level pointer indirection only becomes an issue in unsafe Rust, where they can be used. This decision aligns closer to the MISRA rule rationale.

```
1 struct S {  
2     a: *const *const *const i32,  
3 }  
4  
5 fn main() {}
```

Listing 55: Rust Code Example for guideline 11.3.2

Rule 11.6.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rust enforces RAII, declaring variables requires initialization before they can be used on any control flow path ('Rust Reference: Declaration statements p. 176/432', 'Rust Reference: Variables p. 375/432').

```
1 use std::mem::MaybeUninit;  
2  
3 fn main() {  
4     // Create an uninitialized variable  
5     let mut uninitialized: MaybeUninit<i32> = MaybeUninit::uninit();  
6  
7     unsafe {  
8         // Initialize the variable  
9         uninitialized.as_mut_ptr().write(42);  
10  
11        // Read the value  
12        let value = uninitialized.assume_init();  
13        println!("Initialized value: {}", value);  
14    }  
15 }
```

Listing 56: Rust Code Example for guideline 11.6.1

Rule 11.6.2

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: MaybeUninit can be used to initialize an empty value and read it from it within an unsafe context.

```

1 use std::mem::MaybeUninit;
2
3 fn main() {
4     let x: MaybeUninit<u32> = MaybeUninit::uninit();
5
6     let y = unsafe { x.assume_init() };
7     println!("Value: {}", y);
8 }

```

Listing 57: Rust Code Example for guideline 11.6.2

Rule 11.6.3

Analysis Result

(C3) Rule can be dropped completely

Comment: In Rust values of implicitly-specified enumeration constants are checked for uniqueness during compile time (see example).

```

1 enum MyEnum {
2     Variant1 = 1, // Implicitly assigned value 0
3     Variant2,     // Implicitly assigned value 2
4     // Compilation error: Discriminant value '2' is assigned more
5     // than once
6     Variant3 = 2, // Explicitly assigned value 2
7 }
8
9 fn main() {
10     // Use the enum variants
11     let v1 = MyEnum::Variant1 as i32;
12     let v2 = MyEnum::Variant2 as i32;
13     let v3 = MyEnum::Variant3 as i32;
14
15     assert_eq!(v1, 1);
16     assert_eq!(v2, 2);
17     assert_eq!(v3, 1);
18 }

```

Listing 58: Rust Code Example for guideline 11.6.3

Rule 12.2.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not have Bitfields or an equivalent.

Rule 12.2.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not have bit-fields or an equivalent.

Rule 12.2.3

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not have bit-fields or an equivalent.

Rule 12.3.1

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: Rust's `union` implementation mitigates risks of dropped memory regions or implicit write operations, but this behaviour can explicitly be added to unions. Therefore the rule can be applied to Rust as well ('Rust Reference: Initialization of a Union p.95/432').

Rule 13.1.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust has no direct equivalent to inheritance and virtual inheritance. Due to strict typing, runtime polymorphism is possible in a different, explicit way, without similar problems like diamond hierarchies (see example).

```
1 trait Animal {  
2     fn speak(&self);  
3 }  
4  
5 struct Dog;  
6  
7 impl Animal for Dog {  
8     fn speak(&self) {
```

```

9     println!("Woof!");
10 }
11 }
12
13 struct Cat;
14
15 impl Animal for Cat {
16     fn speak(&self) {
17         println!("Meow!");
18     }
19 }
20
21 fn main() {
22     let animals: Vec<Box<dyn Animal>> = vec![Box::new(Dog), Box::
        new(Cat)];
23
24     for animal in animals {
25         animal.speak();
26     }
27 }

```

Listing 59: Rust Code Example for guideline 13.1.1

Rule 13.1.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: As Rust does not have virtual classes, the behaviour is less ambiguous (see example 13.1.1).

Rule 13.3.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust's equivalent of virtual functions are trait implementations. Trait implementations are explicit (impl block). This eliminates the rule's rationale for accidental overrides through Rust's language design (see example).

```

1 // Define a trait with a default implementation for a method
2 trait Greet {
3     fn greet(&self) {
4         println!("Hello from the default implementation!");
5     }
6 }
7

```

```

8 // Implement the trait for a struct without overriding the default
  method
9 struct Person;
10 impl Greet for Person {}
11
12 // Implement the trait for another struct and override the default
  method
13 struct Dog;
14 impl Greet for Dog {
15     // Overwrite has no specifier
16     fn greet(&self) {
17         println!("Woof! Woof!");
18     }
19 }
20
21 fn main() {
22     let person = Person;
23     let dog = Dog;
24
25     // Call the greet method on both instances
26     person.greet(); // Outputs: Hello from the default
                       implementation!
27     dog.greet();    // Outputs: Woof! Woof!
28 }

```

Listing 60: Rust Code Example for guideline 13.3.1

Rule 13.3.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not support default arguments in function signatures, the extend of confusing override behaviour is what is described in rule 13.3.1.

Rule 13.3.3

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Rust does not allow unnamed function arguments. Arguments must be named and explicitly typed ('Rust Programming Language: Functions p. 56/626').

```

1 trait T {
2     fn t(a: i32);
3 }
4

```

```

5 struct A;
6 impl T for A {
7     fn t(a: i32) {
8
9     }
10 }
11
12 struct B;
13 impl T for B {
14     fn t(b: i32) {
15
16     }
17 }
18
19 fn main() {
20
21 }

```

Listing 61: Rust Code Example for guideline 13.3.3

Rule 13.3.4

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not have a notion of virtual pointer-to-member pointers ('Rust Reference: Raw Pointers p. 289/432').

Rule 14.1.1

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Rust allows both public and non-public members simultaneously. Therefore the rule applies, even if nuances such as static keyword and protected functions are different in Rust (see example).

```

1 pub struct MyStruct {
2     pub public_field: i32,
3     private_field: i32,
4 }
5
6 impl MyStruct {
7     pub fn new(public_value: i32, private_value: i32) -> MyStruct {
8         MyStruct {
9             public_field: public_value,
10            private_field: private_value,
11        }
12    }
13 }

```

```

12     }
13
14     pub fn get_private_field(&self) -> i32 {
15         self.private_field
16     }
17 }
18
19 fn main() {
20     let my_struct = MyStruct::new(10, 20);
21
22     println!("Public field: {}", my_struct.public_field);
23     println!("Private field: {}", my_struct.get_private_field());
24 }

```

Listing 62: Rust Code Example for guideline 14.1.1

Rule 15.0.1

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: Rationale is to address vulnerabilities of “Rule of Zero”, “Rule of Five”, and C++ 11 “Rule of Three”. Rust handles these special member functions via the special traits: `Copy`, `Clone`, `Drop`. Intricacies of C++ implementations of special member functions are different in Rust. This rule applies in a general sense, but would require careful evaluation to apply to Rust.

Rule 15.0.2

Analysis Result

(C3) Rule can be dropped completely

Comment: In Rust, move semantics are the default behaviour for types not implementing the `Copy` trait. The `Copy` trait has an explicit application rule without ambiguity, and forces implementation of the `Clone` trait as well (see example).

```

1  #[derive(Debug)]
2  struct MyStruct {
3      x: i32,
4      y: i32,
5  }
6
7  // Clone trait implementation with explicit signature
8  impl Clone for MyStruct {
9      fn clone(&self) -> MyStruct {
10         MyStruct {

```

```

11         x: self.x,
12         y: self.y,
13     }
14 }
15 }
16
17 // Implementing Copy, requires Clone trait
18 impl Copy for MyStruct {}
19
20 fn main() {
21     let a = MyStruct { x: 10, y: 20 };
22     let b = a; // a is copied to b using the Copy trait
23
24     // Both a and b can be used independently, without
25     // implementation
26     // of the Copy trait, a would be moved to b, resulting in a
27     // compilation error
28     println!("a: {:?}", a); // Outputs: a: MyStruct { x: 10, y: 20 }
29     println!("b: {:?}", b); // Outputs: b: MyStruct { x: 10, y: 20 }
30 }

```

Listing 63: Rust Code Example for guideline 15.0.2

Rule 15.1.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust object construction and destruction is done differently. For construction, typically an associated function (per convention named new) without self (or this) reference is used, to return a value of the type being constructed. The problem addressed by this rule does not occur. Destruction requires implementation of the Drop trait, which has no access to a dynamic trait implementation and is called automatically.

```

1 trait Animal {
2     fn speak(&self);
3 }
4
5 struct Dog {
6     name: String,
7 }
8
9 impl Animal for Dog {
10     fn speak(&self) {
11         println!("{}", self.name);
12     }
13 }
14
15 impl Dog {

```

```

16 // Associated function to create a new Dog
17 pub fn new(name: &str) -> Dog {
18     Dog {
19         name: name.to_string(),
20     }
21 }
22 }
23
24 impl Drop for Dog {
25     fn drop(&mut self) {
26         println!("Dropping Dog named: {}", self.name);
27     }
28 }
29
30 fn main() {
31     // alternative direct instantiation
32     let my_dog = Dog {
33         name: "Buddy".to_string(),
34     };
35     let my_dog: Box<dyn Animal> = Box::new(Dog::new("Buddy"));
36     my_dog.speak();
37     // my_dog goes out of scope here, and Drop is called
38     // automatically
39 }

```

Listing 64: Rust Code Example for guideline 15.1.1

Rule 15.1.2

Analysis Result

(C3) Rule can be dropped completely

Comment: Rust does initialization in an explicit way. A comparable mechanism to class inheritance in Rust is Supertraits. Due to explicit typing and initialization, those do not exhibit the same ambiguous behaviour when initialized. ('Rust Reference: Supertraits 108/432').

Rule 15.1.3

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust has only explicit initialization, without implicit conversions. The closest equivalent is the `tuple struct`, but these are not affected by this rule (see example).

```

1 struct SomeStruct(i64);
2
3 impl SomeStruct {
4     // Associated function to create a new SomeStruct
5     pub fn new(value: i64) -> SomeStruct {
6         SomeStruct(value)
7     }
8
9     // Method to get the value
10    pub fn value(&self) -> i64 {
11        self.0
12    }
13 }
14
15 fn main() {
16     let my_struct = SomeStruct::new(42);
17     println!("The value is: {}", my_struct.value());
18 }

```

Listing 65: Rust Code Example for guideline 15.1.3

Rule 15.1.4

Analysis Result

(C3) Rule can be dropped completely

Comment: On function entry, space for Rust objects is reserved on the stack. Initialization has to be finished before access, this is enforced during compile time ('Rust Reference: Variables p. 375/432').

Rule 15.1.5

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rationale behind this rule is unclear behaviour under special overload resolution rules. Rust does not have an overload mechanism, therefore no need to address this.

Dir 15.8.1

Analysis Result

(C3) Rule can be dropped completely

Comment: This rule is concerned with undefined behaviour during copy and move assignments, when assigning to self. When providing user-defined implementations of Rust's `Copy` trait, the `Clone` trait is required. Furthermore, implementing the `Copy` trait is not possible for heap-allocated types (like `Box`). Here only a `Clone` trait implementation is possible. Due to Rust's move mechanics, self-assignments is not an issue (see example).

```
1  #[derive(Debug)]
2  struct MyStruct {
3      x: Box<i32>,
4  }
5
6  // Implementing Clone manually
7  impl Clone for MyStruct {
8      fn clone(&self) -> MyStruct {
9
10         // Valid handling of Box type (heap allocated memory) is to
11         // clone the value inside the Box
12         MyStruct { x: self.x.clone() }
13     }
14 }
15
16 // Implementing Copy would result in a compilation error:
17 // the trait 'Copy' cannot be implemented for this type (MyStruct,
18 // because it uses Box)
19 impl Copy for MyStruct {}
20
21 fn main() {
22     let a = MyStruct { x: Box::new(10) };
23     let b = a.clone(); // a is cloned, because Box can not
24                       // implement Copy
25
26     // Both a and b can be used independently
27     println!("a: {:?}", a); // Outputs: a: MyStruct { x: 10, y: 20 }
28     println!("b: {:?}", b); // Outputs: b: MyStruct { x: 10, y: 20 }
```

Listing 66: Rust Code Example for guideline 15.8.1

Rule 16.5.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust implements operators via traits. Custom implementations (overloading) `&&` and `||` is not possible: <https://doc.rust-lang.org/std/ops/index.html>.

Rule 16.5.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust provides no operator trait for `&`.

Rule 16.6.1

Analysis Result

(C3) Rule can be dropped completely

Comment: The rationale behind this rule is unexpected implicit type conversions for symmetrical member functions. For Rust, overriding a (symmetric) operator by implementing it's trait, is explicitly typed. Therefore this problem does not arise (see example).

```
1 use std::ops::Add;
2
3 #[derive(Debug)]
4 struct MyStruct {
5     x: i32,
6     y: i32,
7 }
8
9 // Implementing the Add trait for MyStruct
10 impl Add for MyStruct {
11     type Output = MyStruct;
12     // Strongly typed
13     fn add(self, other: MyStruct) -> MyStruct {
14         MyStruct {
15             x: self.x + other.x,
16             y: self.y + other.y,
17         }
18     }
19 }
20
21 fn main() {
22     let a = MyStruct { x: 10, y: 20 };
23     let b = MyStruct { x: 30, y: 40 };
24     let c = a + b; // Using the overloaded + operator
25     assert_eq!(c.x, 40);
26     assert_eq!(c.y, 60);
27 }
```

Listing 67: Rust Code Example for guideline 16.6.1

Rule 17.8.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not have overloaded functions. The rationale behind this rule is developer confusion, because overload resolution in conjunction with explicit specialization might lead to unexpected results. Rust restricts specialization, function names are still unique (see example and ‘Rust Reference: Generic functions p. 79/432’).

```
1 // Using this generic function would result in a compilation error
2 // error[E0428]: the name 'generic' is defined multiple times
3 // fn generic<T>(arg: T) -> () {}
4
5 fn generic<i64>(arg: i64) -> () {}
6 fn main() {
7     let s = generic::
```

Listing 68: Rust Code Example for guideline 17.8.1

Rule 18.1.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not have an exception model like C++. Instead errors are categorized in recoverable and unrecoverable errors. For Rust, the special type **Result** exists, to catch expected runtime errors. Unexpected runtime errors will be caught by a panic handler. **Results** can have arbitrary types, including referenced memory. Due to borrow checks, this rule might possibly not apply, but in general the error handling is different (‘Rust Programming Language: Error handling p. 196/626’).

Rule 18.1.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rationale is preventing implementation-defined program termination, based on C++ specific exception handling. As Rust does not have a similar exception mechanism, no application of this rule is obvious (‘Rust Programming Language: Error handling p. 196/626’).

Rule 18.3.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust employs a panic handler, but its purpose is only with regard to unrecoverable errors. Exceptions in C++ are purposed to handle all types of errors. However, panics are non-recoverable. Therefore the concept of catchable exceptions does not exist in Rust ('Rust Programming Language: Error handling p. 196/626').

Rule 18.3.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust's `Result` type provides the possibility of arbitrary error types. Due to explicit types, the rationale (losing information due to slicing) behind this rule is not satisfied. Also, as stated in rule 18.3.1, Rust has no exception types, making this rule not applicable.

Rule 18.3.3

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: This problem can not occur in Rust, as initialization before access is enforced at compile time ('Rust Reference: Variables p. 375/432').

Rule 18.4.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rationale is prevention of implementation-defined termination if an exception occurs during call of a destructor. As Rust handles release of resources automatically based on lifetimes, this is not required in general. Additionally, the only way to return an error message from functions is either through the type system or a panic. There are no "surprises".

Rule 18.5.1

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: Rust has no equivalent to `noexcept` therefore there is no automatic passing of errors to calling functions. Different methods in Rust exist to propagate errors, some of which may lead to accidental omission by the developer (see example).

```
1 fn returns_err() -> Result<(), ()> {  
2     Err(())  
3 }  
4  
5  
6 fn main() -> Result<(), ()> {  
7     // ? operator propagates error, if the Result does yield an Err  
8     let res = returns_err()?;  
9     // direct unwrap of Result, if it yields Err, this panics  
10    let res2 = returns_err().unwrap();  
11    if let Ok(val) = returns_err() {  
12        // handle only Ok(()), missing Error handling  
13    }  
14    Ok(())  
15 }
```

Listing 69: Rust Code Example for guideline 18.5.1

Rule 18.5.2

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Applies, as Rust has functions to terminate execution (see example).

```
1 use std::process;  
2  
3 fn main() {  
4     println!("This will terminate the program.");  
5     process::exit(1); // Exit with status code 1  
6     println!("Unreachable location");  
7 }
```

Listing 70: Rust Code Example for guideline 18.5.2

Rule 19.0.1

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: Rust uses different control flow syntax, but typos, as outlined in the rationale, can still cause the same type of error (see example).

```
1 fn main() {
2     platform_specific_function();
3 }
4
5 // Correct cfg attribute
6 #[cfg(target_os = "windows")]
7 fn platform_specific_function() {
8     println!("Running on Windows");
9 }
10
11 // Typo in cfg attribute (target_os is misspelled as target_osx)
12 #[cfg(target_osx = "linux")]
13 fn platform_specific_function() {
14     println!("Running on Linux");
15 }
16
17 // Fallback function for unsupported platforms
18 #[cfg(not(any(target_os = "windows", target_os = "linux")))]
19 fn platform_specific_function() {
20     println!("Running on an unsupported platform");
21 }
```

Listing 71: Rust Code Example for guideline 19.0.1

Rule 19.0.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Some behaviour of Rust macros is improved, e.g., strict type checks (see example) and a different syntax than function calls. Rust procedural macros generate Rust code, including function-like macros. The resulting code is type checked and compiled as if a human would have written it. The rationale of the rule does not apply ('Rust Reference: Procedural Macros p. 47/432').

```
1 macro_rules! add {
2     ($a:expr, $b:expr) => {
3         $a + $b
4     };
5 }
6
```

```

7 fn main() {
8     let result = add!(1, 2);
9     // compilation error: cannot add 'char' to 'char'
10    //let _ = add!('a', 'b');
11    assert_eq!(result, 3);
12 }

```

Listing 72: Rust Code Example for guideline 19.0.2

Rule 19.0.3

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Rationale is readability, thus this rule naturally applies (see example).

```

1 fn defined_too_early() -> i32 {
2     42
3 }
4
5 use std::marker::PhantomData;
6
7 fn main() {
8     let _ = PhantomData::<i32>;
9     let _ = defined_too_early();
10 }

```

Listing 73: Rust Code Example for guideline 19.0.3

Rule 19.0.4

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: The most direct equivalent to preprocessor macros in Rust are attributes. Different categories of attributes exist, that mimic C++ macros. A difference in method can be observed, as Rust attributes can be used to activate or deactivate code based on conditions (see example), but do not have an equivalent to `#undef`. A complete analysis checking underlying grammars for equivalence is out of scope. The rationale behind the rule is to prevent developer confusion, and for the case of attributes in Rust, it can be confusing to see exactly how code behaves if attribute configuration is used extensively ('Rust Reference: Attributes p. 141/432').

Rule 19.1.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: This rule addresses undefined behaviour of the defined preprocessor operator, thus is not applicable to Rust.

Rule 19.1.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Attributes in Rust are directly applied to a code object, stretching the scope of effect over multiple files (or even code objects in the same file) is not possible ('Rust Reference: Attributes p. 141/432').

Rule 19.1.3

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: If configuration attributes are used to configure the code base, non-defined features cause a compilation warning, which can be promoted to an error (see example).

```
1 // Non-existing feature: compilation WARNING: unexpected 'cfg'
  condition value
2 // compiler note: expected values for 'feature' are: ['list', 'of',
  'existing', 'features']
3 #[cfg(feature = "feature_does_not_exist")]
4 fn feature_function() {
5     println!("Feature is enabled!");
6 }
7
8 fn main() {
9     // compilation error: cannot find function 'feature_function'
    in this scope
10     feature_function();
11 }
```

Listing 74: Rust Code Example for guideline 19.1.3

Rule 19.2.1

Analysis Result

(C3) Rule can be dropped completely

Comment: Rationale is to prevent confusing behaviour and unexpected re-definition due to collisions in complex include structures. Due to strict import rules and compile time checks against redefinition, the problems addressed by this rule are solved at compile time in Rust (see example).

```
1 use std::marker::PhantomData;  
2 // Compilation error: the name 'PhantomData' is defined multiple  
   times  
3 use std::marker::PhantomData;  
4  
5 fn main() {}
```

Listing 75: Rust Code Example for guideline 19.2.1

Rule 19.2.2

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: As Rust does not use filenames, but rather crate names, and instead of the `include` directive the `use` directive, the guideline is not directly applicable. In general some restriction on valid import syntax in Rust is required (see reasoning guideline 19.2.3).

Rule 19.2.3

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: File names are restricted by OS. The sequences given in the rule would also lead to invalid Rust import names.

Rule 19.3.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rationale for this rule is unspecified order of evaluation for these operators. Rust does not have a direct equivalent. Macro invocation in Rust expands the macro at compile time, similar to the C++ preprocessor, but offers unique fragment-specifiers to match a Rust syntax fragment. To satisfy this rule, Rust macros would be required to be more concise, reducing ambiguity. In the given example, an equivalent Rust macro call is shown. Due to missing language features, to receive similar functionality to the `#` and `##` operators, the example given for the macro invocation heavily relies on function calls, solving the task at runtime instead of compile time (see example, ‘Rust Reference: Macros p. 35/432’ to ‘p. 42/432’).

```
1 macro_rules! example_macro {  
2     ($a:ident, $b:ident) => {{  
3         // In Rust version 1.82.0, the only macro as powerful as  
4         // the # and ## operator in C++ is concat_ids!() which  
5         // is unstable.  
6         // The concat_ids!() macro is used to concatenate two  
7         // identifiers into a single identifier.  
8  
9         // One possibility to combine the identifiers is given here  
10        // as a macro, but in reality offers only disadvantages  
11        // over a function call  
12        let a_str = $a.to_string();  
13        let b_str = $b.to_string();  
14        let combined_str = format!("{}", a_str, b_str);  
15        combined_str.parse::<i32>().unwrap()  
16    }};  
17 }  
18  
19 fn main() {  
20     let x = 10;  
21     let y = 1;  
22     let result = example_macro!(x, y);  
23     assert_eq!(result, 101);  
24 }
```

Listing 76: Rust Code Example for guideline 19.3.1

Rule 19.3.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: See explanation and example rule 19.3.1.

Rule 19.3.3

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rationale is differing expanding behaviour of parameters, depending on the context. As with rule 19.3.2, Rust does not need to address this due to rule 19.3.1.

Rule 19.3.4

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: The rationale is to prevent unexpected behaviour due to textual replacement in C++ macros, instead of handling macro arguments like parameters to a function call. In Rust, meta-variables are used to match macro parameters, resulting in behaviour that resembles type checks. Where in C++ operator precedence and textual replacement might lead to unexpected behaviour, Rust matches operands as expressions (see example). Rust's mechanism adds safety for the given rationale.

```
1 macro_rules! add_multiply {
2     // Metavariable expression
3     ($a:expr, $b:expr) => {
4         $a * $b
5     };
6 }
7
8 fn ident(num: i32) -> i32 {
9     num
10 }
11 fn main() {
12     let x = 2;
13     let y = 3;
14
15     // Without parentheses, the macro would produce unexpected
16     // results in C, but in Rust
17     // the parameters are matched as one indivisible expression
18     // token, and evaluated correctly
19     // C++: this macro would expand to 1 + 2 * 3 = 7
20     // Rust: 1+2 is expression $a, 3 is expression $b, both are
21     // evaluated separately
22     let result1 = add_multiply!(ident(1) + 2, 3);
23     println!("Result without parentheses: {}", result1); // Output:
24     9
25
26     // With parentheses, the calculation would be:
27     // C++: 3 * 3 = 9
28     // Rust: 3 * 3 = 9
29     let result2 = add_multiply!(ident(3), (3));
```

```

26 |     println!("Result with parentheses: {}", result2); // Output: 9
27 | }

```

Listing 77: Rust Code Example for guideline 19.3.4

Rule 19.3.5

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rationale is undefined behaviour due to textual replacement resulting in valid code with undefined semantics. As Rust has a different macro syntax and semantic, this rule is specific to C++ ('Rust Reference: Macros p. 35/432').

Rule 19.6.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rationale is implementation-defined behaviour of the C++ compiler. Rust does not have an official standard. From a technical perspective, all features can be considered to be implementation-specific. However, the Rust foundation maintains a reference <https://doc.rust-lang.org/reference/>. To keep the spirit of the MISRA rule, we categorize the item as class C1, since the `#pragma` is not available within Rust as a concept.

Rule 21.2.1

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Rationale is undefined behaviour in these stdlib functions.

Rule 21.2.2

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Does naturally not apply.

Rule 21.2.3

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Does naturally not apply.

Rule 21.2.4

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Does naturally not apply.

Rule 21.6.1

Analysis Result

(C6) Rule still required (safe Rust)

Comment: Naturally applies to Rust, as dynamic memory exists in Rust as well. General problems like resource exhaustion cannot be mitigated in general.

```
1 fn main() {  
2     unsafe {  
3         // Allocate memory for an integer on the heap using Box  
4         let boxed_value = Box::new(42);  
5  
6         // Convert the Box into a raw pointer  
7         let ptr: *mut i32 = Box::into_raw(boxed_value);  
8  
9         // Intentionally forget to deallocate the memory, causing a  
10        memory leak  
11        // Box::from_raw(ptr); // This line is commented out to  
12        create a memory leak  
13  
14        // Access the allocated memory  
15        println!("Value: {}", *ptr);  
16  
17        // Normally, you should deallocate the memory to avoid a  
18        memory leak  
19        // Box::from_raw(ptr); // This would convert the raw  
20        pointer back into a Box and deallocate it  
21    }  
22 }
```

Listing 78: Rust Code Example for guideline 21.6.1

Rule 21.6.2

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: Rust's lifetime management guarantees this. When not specified, lifetime elision is used. Additionally explicit lifetime annotations are required by the compiler, where automatic elision is not possible. Variables going out of scope will call their destructor equivalent automatically (provided by the `Drop` trait implementation). A natural adaptation of this rule to Rust should include rules to address explicit `Drop` implementations by a user ('Rust Reference: Lifetime elision p. 340/432', see example for rule 21.6.4).

Rule 21.6.3

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rationale is that specific handling of dynamic memory leads to potential issues with alignment, object lifetimes, and user implementations of `new` or `delete` operators, all resulting in undefined behaviour. Rust does not have such specific memory management in non-unsafe code. For dynamic memory allocation in a safe context, only specific types like `Box` or `Vec` types exist ('Rust Reference: Memory allocation and lifetime p. 374/432'). See rule 21.6.2 for the `Drop` trait.

Rule 21.6.4

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust does not allow global overwrite of deallocation logic like in C++. See rule 21.6.2 for the `Drop` trait.

```
1 struct MyStruct {  
2     data: *mut i32,  
3 }  
4  
5 impl MyStruct {  
6     fn new(value: i32) -> Self {  
7         // Allocate memory on the heap  
8         let data = Box::into_raw(Box::new(value));  
9         MyStruct { data }
```

```

10     }
11 }
12
13 impl Drop for MyStruct {
14     fn drop(&mut self) {
15         // Incorrect: Forgetting to deallocate the memory
16         // This will cause a memory leak
17         println!("Dropping MyStruct, but not deallocating memory");
18     }
19 }
20
21 fn main() {
22     let my_struct = MyStruct::new(42);
23     // When my_struct goes out of scope, the drop method will be
24     // called,
25     // but the memory allocated on the heap will not be deallocated
26     .
27 }

```

Listing 79: Rust Code Example for guideline 21.6.4

Rule 21.6.5

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Full type definition is required in Rust before usage, this includes manual deletion via the `Drop` trait implementation (see explanation for rule 0.2.1 and rule 21.6.4).

Rule 21.10.1

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Rule naturally not applicable, as this is specific to the C++ `stdlib`.

Rule 21.10.2

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Rule naturally not applicable, as this is specific to the C++ `stdlib`. These functions allow bypassing the normal function return mechanism.

This rule addresses concerns of safety standards like ISO 26262 (Part 6, Table 6), where “single-entry single-exit” principle is encouraged.

Rule 21.10.3

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Rule naturally not applicable, as this is specific to the C++ `stdlib`. Signal handling is per the rationale defined in ISO 9899.

Rule 22.3.1

Analysis Result

(C5) Rust-specific adaptation potentially required

Comment: Rationale is the runtime evaluation of `assert`, when compile time evaluation with `static_assert` would be possible. Rust provides the `assert!` macro, that evaluates const expressions at compile time, and non-const expressions at runtime. The underlying rationale of this rule is addressed. Due to the duality of the macro, developer expectations of when the `assert` macro is evaluated might not be clear at a glance (see example).

```
1 fn main() {  
2     let y = 5;  
3     let z = 6;  
4     let runtime_assert_result = assert!(y != z);  
5  
6     const x: i32 = 5;  
7     // This will not compile, as the assertion evaluates to false  
8     // during compile time  
9     const COMPILE_TIME_ASSERT_RESULT: () = assert!(x == 4);  
}
```

Listing 80: Rust Code Example for guideline 22.3.1

Rule 22.4.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rust uses the `Result` type to handle errors.

Rule 23.11.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rationale is avoiding creation of raw pointers as an intermediary step. Rust without unsafe code does not allow those raw pointers ('Rust Reference: Raw Pointers p. 289/432').

Rule 24.5.1

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Rules does naturally not apply.

Rule 24.5.2

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Rationale is undefined behaviour in cases of memory overlap, potential overlap, is not trivially copyable. With regard to `memcpy`, refer to rule 8.18.1 for more insight. Rust does not have a direct equivalent to `memcpy`.

Rule 25.5.1

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Does naturally not apply to Rust.

Rule 25.5.2

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Does naturally not apply to Rust.

Rule 25.5.3

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Does naturally not apply to Rust.

Rule 26.3.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Does naturally not apply to Rust.

Rule 28.3.1

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Does naturally not apply, as the rationale is implementation-defined copy behaviour of special template parameters.

Rule 28.6.1

Analysis Result

(C4) Rule still required (unsafe Rust)

Comment: Rationale is the `std::move` function not moving content of const objects. This specific rule is not applicable to Rust, as moved values are inaccessible at the original side in Rust (see rule 15.0.2).

Rule 28.6.2

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Rationale is idiomatic use of perfect forwarding in C++, consistently preserving the value category. There is no clear equivalent in Rust

that matches this exact rule. In general the ownership model addresses the underlying problem differently ('Rust Programming Language: Understanding Ownership' p. 75/626).

Rule 28.6.3

Analysis Result

(C2) Not applicable: C++ Feature does not exist in Rust

Comment: Potentially moved refers to objects created by calling `std::move`, `std::forward`, or an equivalent `static_cast`. Similar to rule 28.6.2, Rust's ownership mechanism addresses the underlying issues differently (see rule 28.6.2).

Rule 28.6.4

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Rationale is preventing the programmer from ignoring an unexpected empty result, when the goal might have been to purge by calling `std::erase` in conjunction.

Rule 30.0.1

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Rationale is undefined, unspecified, and implementation-defined behaviour associated with those functions. Rule does naturally not apply.

Rule 30.0.2

Analysis Result

(C1) Not applicable: C++ Standard Library Usage Restriction

Comment: Rationale is specifics of the C FILE * abstraction.